Distributed Systems

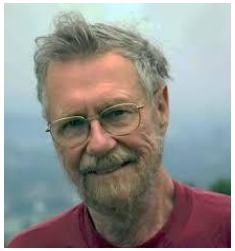
Background

CUHK

Distributed computing

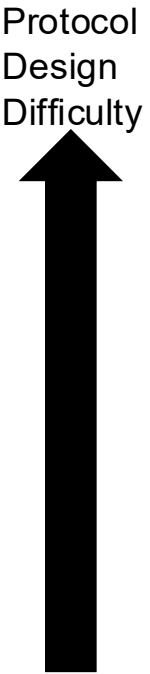
- Availability vs Scalability vs Consistency
- System Model of a solution is captured by multiple dimensions:
  - Communication
  - Fault
  - Consistency
- We design algorithms (protocols) / build systems that satisfy a particular system model

"His foundational work on concurrency primitives (such as the semaphore), concurrency problems (such as mutual exclusion and deadlock), reasoning about concurrent systems, and self-stabilization comprises one of the most important supports upon which the field of distributed computing is built. No other individual has had a larger influence on research in principles of distributed computing."



Communication Model

Asynchronous	No upper bound on the delay Δ of messages or delay Φ of processors But != message lost	$\Delta/\Phi=(0,\infty)$
Partial Synchrony	There is an unknown upper bound on Δ/Φ	Δ/Φ = (0,X)
Semi Synchrony	Assume there is known probability distribution on Δ/Φ	Δ/Φ ~ N(x,y)
Weak Synchrony	Δ/Φ is varying, but it grows at most polynomials in timeout t	Δ/Φ = 2t
Full Synchrony	There is a known upper bound on Δ/Φ	Δ/Φ = (0,10ms)

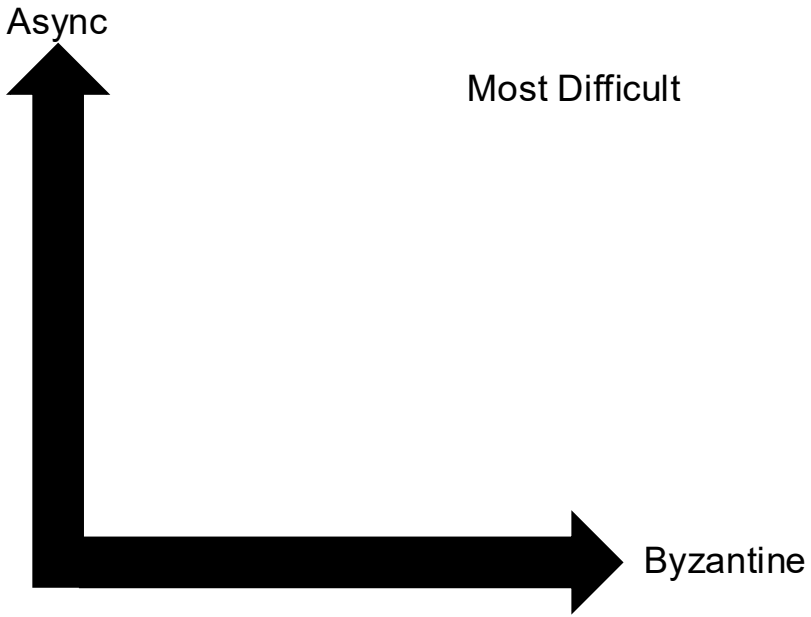


Fault Model

Crash-Fault

Byzantine Fault

Protocol Design Difficulty


Async
Most Difficult
Byzantine

Measuring the goodness of a distributed algorithm

- Time is measured by
  - **Message complexity**
    - Number of messages sent (=received) as a function of n
      - n: number of nodes
      - E.g., $O(n^2)$ vs $O(n)$

Correctness

- Safety properties
  - Properties that shall hold true throughout the lifetime of system
- Liveness properties
  - The goal
  - Eventually this property holds ➔ goal achieved

Safety

- Ensure a predicate P evaluated on the states **always true**
  - (or have a way to go back to true on fault happens)
- E.g., Mutual Exclusion Problem
  - Predicate P is: $N_{cs} <= 1$ (N_{cs} is the number of processes in CS)

How to prove safety?

- E.g.,
  - Show that none of the states S_0 ➔ S_1 ➔ … S_f violates the predicate

Liveness

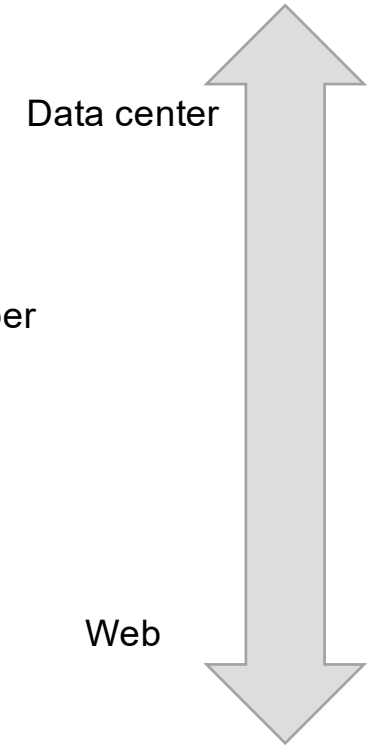
- Desired properties **eventually hold**
- Not easy to prove the future by examining a finite prefix of state history
  - One option: prove using Logic (specifically, temporal logic) [L. Lamport. TLA+: The temporal logic of actions. ACM Trans. Prog. Lang. Syst., 1994], model checking
- E.g., “Progress” in Critical Section Problem
  - Once a process declares it wants to go into critical section (CS), **eventually** it can enter
    - In other words, a solution that violates liveness (progress) would make a process ‘hangs’ forever
- E.g., Termination
  - Starting from initial state, must eventually **terminate**

Liveness

- E.g., “Consensus” Problem
  - Once the processes start the algorithm
  - Liveness: Eventually they agree on the value
  - This wouldn’t be a safety property because it is normal for the nodes disagree while the protocol is running

High-level communication methods between distributed nodes

- RPC
  - E.g., Spark, Tensorflow, Zookeeper
- REST
- GraphQL
- Websocket



Fewer network problems
More efficient

More network problems
Less efficient

Remote Procedure Call

- Network/Socket programming: lower level
- RPC programming: higher level
  - Call interface is defined by a language agnostic description language (e.g., .proto)
  - You define the high-level function calls (e.g., createSales)
  - RPC library generates stubs for client and server
  - The client code can invoke a remote function as a normal function
    - The library will take care of the networking and **(de)serialization** issues for you
- Usually for communication between nodes with known schema/API during the development time of a distributed system
  - E.g., Hadoop, Zookeeper, Tensorflow, Spark